Swytchcode Integration Documentation — MindCommit

> How Swytchcode powers every external API call in MindCommit

Table of Contents

---

1. What is Swytchcode

Swytchcode is a **self-hosted AI agent execution layer** that sits between AI agents (like MindCommit) and production APIs. It provides one runtime for:

- **Authentication** — OAuth, API keys, bearer tokens resolved automatically
- **Schema Validation** — Every API call validated before execution
- **Policy Enforcement** — Define what the agent can/cannot do
- **Retries** — Automatic retry with exponential backoff

- **Idempotency** — Prevent duplicate mutations
- **Audit Trail** — Full log of every external action

Official resources:

- Website: https://www.swytchcode.com/
- Documentation: https://docs.swytchcode.com/
- API Catalog: https://www.swytchcode.com/apis
- GitHub: https://github.com/swytchcodehq

The CLI binary is `swytchcode`, with `swy` as the shorter alias.

---

2. Why MindCommit Uses Swytchcode

MindCommit connects to **10+ external APIs** to synchronize knowledge sources, send answers, and enable team access to the Elon Musk Knowledge Twin. Without Swytchcode, we would need:

❌ Manual OAuth flow for Google Drive, Gmail, Notion, YouTube ❌ Custom retry logic for each API ❌ Separate API key management per service ❌ Hand-written schema validation per endpoint ❌ No audit trail of agent actions ❌ No policy controls (agent could delete data accidentally)

With Swytchcode:

✅ One `swy exec` call handles everything ✅ `tooling.json` defines what tools exist ✅ `policies.json` defines what the agent is allowed to do ✅ Auth, retries, idempotency are automatic ✅ Full audit trail with `swy audit` ✅ Dry-run mode for safe testing

---

3. Installation & Setup

Install Swytchcode CLI globally

```
npm install -g swytchcode
```

Verify installation:

```
swy --version
```

Initialize the MindCommit project

```
cd e:\hackthon\nsut\ hack\vibewrite
swy init
```

This creates the `.swytchcode/` directory with project configuration.

Authenticate

```
swy login
```

This opens a browser window for Swytchcode account authentication.

Install all integrations

```
# Knowledge ingestion sources
swy get googledrive
swy get notion
swy get github
swy get youtube
swy get firecrawl

# Communication channels
swy get gmail
swy get slack
swy get telegram
swy get resend

# Utilities
swy get google_calendar
```

Enable specific tools

```
# Google Drive tools
swy add googledrive.list_files
swy add googledrive.get_file
swy add googledrive.download_file

# Notion tools
swy add notion.search_pages
swy add notion.get_page
swy add notion.get_block_children
swy add notion.query_database

# GitHub tools
swy add github.search_repos
swy add github.get_repo
swy add github.list_commits
swy add github.get_file_content

# YouTube tools
swy add youtube.search_videos
swy add youtube.get_video_details
swy add youtube.get_captions

# Gmail tools
swy add gmail.send_email
swy add gmail.list_messages
swy add gmail.get_message

# Slack tools
swy add slack.post_message
swy add slack.list_channels
swy add slack.get_channel_history

# Telegram tools
swy add telegram.send_message
swy add telegram.get_updates

# Resend tools
swy add resend.send_email
```

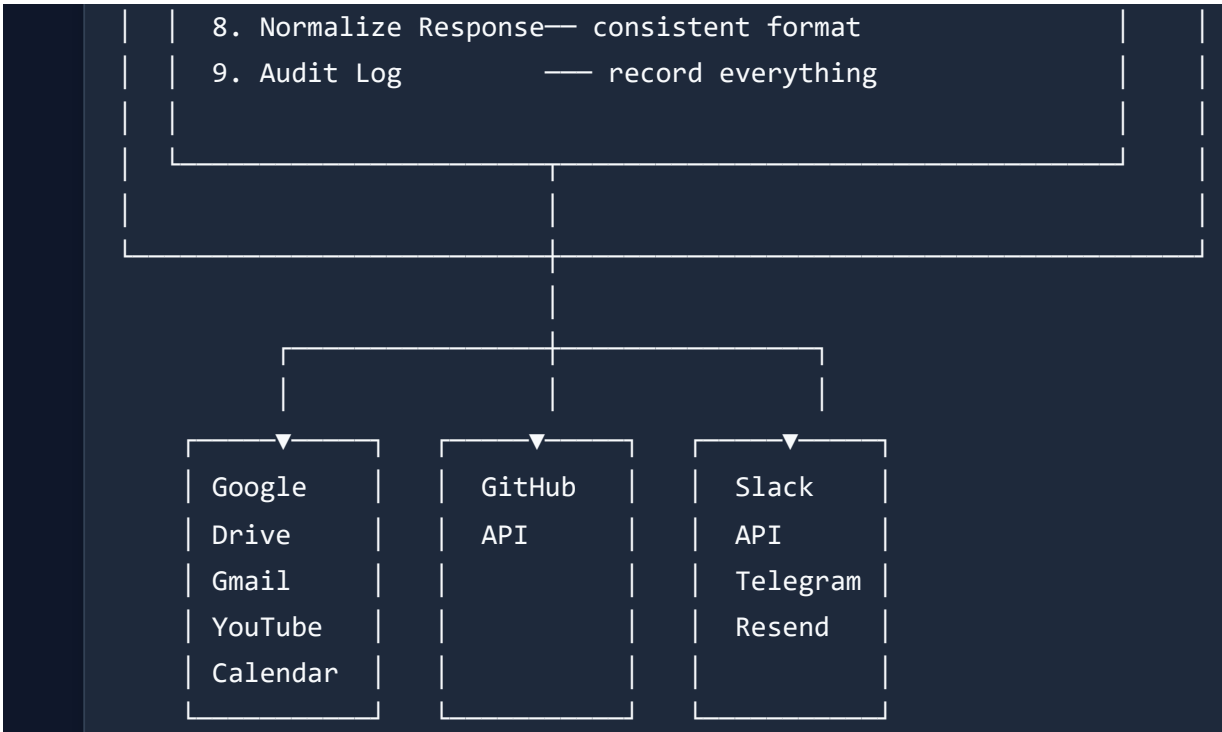
```
# Web scraping tools
swy add firecrawl.scrape_url
swy add firecrawl.crawl_url

# Calendar tools
swy add google_calendar.list_events
swy add google_calendar.create_event
```

Connect provider credentials

```
# Connect Google (Drive, Gmail, Calendar, YouTube)
swy auth connect google

# Connect Notion
swy auth connect notion

# Connect GitHub
swy auth connect github

# Connect Slack
swy auth connect slack

# Verify status
swy auth status
```

---

4. Architecture — How Swytchcode Fits In



Key Point: Every arrow from MindCommit to an external API goes through Swytchcode. There are ZERO direct API calls.

---

5. Execution Pipeline

When MindCommit calls `executeSwytchcodeTool('gmail.send_email', {...})`, here's exactly what happens:

```
Step 1: RESOLVE TOOL
  └─ Swytchcode reads .swytchcode/tooling.json
  └─ Finds 'gmail.send_email' in the enabled tools list
  └─ If not found → error: "Tool not found"

Step 2: VALIDATE INPUT
  └─ Validates the input JSON against the tool's schema
  └─ Checks required fields (to, subject, body)
  └─ If invalid → error: "Invalid input"

Step 3: EVALUATE POLICIES
  └─ Reads .swytchcode/policies.json
  └─ Finds rule: gmail.send_email → "require_approval"
  └─ If denied → error: "Blocked by policy"
  └─ If require_approval → pauses for human review

Step 4: RESOLVE CREDENTIALS
  └─ Swytchcode Managed Auth resolves Google OAuth tokens
  └─ No hardcoded credentials in our code
  └─ Tokens refreshed automatically if expired

Step 5: APPLY EXECUTION POLICY
  └─ Reads manifest.json for retry config
  └─ Sets timeout (30s default)
  └─ Generates idempotency key for mutations

Step 6: EXECUTE API CALL
  └─ Makes the actual Gmail API request
  └─ If fails with 429/500/503 → automatic retry

Step 7: NORMALIZE RESPONSE
  └─ Converts provider-specific response to standard format
  └─ Consistent error handling

Step 8: AUDIT LOG
  └─ Records: tool, input, output, timestamp, latency, status
  └─ Viewable via `swy audit`
```

---

6. All 10 Integrations — Complete Reference

6.1 Google Drive — Knowledge Ingestion

Purpose: Sync approved documents (PDFs, Google Docs) as Knowledge Commits

Tool	Description	Policy
`googledrive.list_files`	List files in Drive folders	Allow
`googledrive.get_file`	Get file metadata	Allow
`googledrive.download_file`	Download file content	Allow

Used in: `src/lib/swytchcode/sync-engine.ts` → `syncGoogleDrive()`

CLI example:

```
swy exec googledrive.list_files --body '{"query":"mimeType=\"application/p
```

6.2 Notion — Structured Notes

Purpose: Pull structured notes, wikis, and databases as Knowledge Commits

Tool	Description	Policy
`notion.search_pages`	Search across Notion workspace	Allow
`notion.get_page`	Get a specific page	Allow
`notion.get_block_children`	Get page content blocks	Allow
`notion.query_database`	Query a Notion database	Allow

Used in: `src/lib/swytchcode/sync-engine.ts` → `syncNotion()`

CLI example:

```
swy exec notion.search_pages --body '{"query":"product roadmap"}'
```

6.3 GitHub — Code Knowledge

Purpose: Retrieve repos, commit messages, documentation for code-related knowledge

Tool	Description	Policy
`github.search_repos`	Search repositories	Allow
`github.get_repo`	Get repo details	Allow
`github.list_commits`	List commit messages	Allow
`github.get_file_content`	Read file content	Allow

Used in: `src/lib/swytchcode/sync-engine.ts` → `syncGitHub()`

CLI example:

```
swy exec github.search_repos --body '{"query":"tesla autopilot"}'
```

6.4 YouTube — Interviews & Talks

Purpose: Extract captions from interviews, lectures, and talks

Tool	Description	Policy
`youtube.search_videos`	Search for videos	Allow
`youtube.get_video_details`	Get video metadata	Allow
`youtube.get_captions`	Extract video captions/transcripts	Allow

Used in: `src/lib/swytchcode/sync-engine.ts` → `syncYouTube()`

CLI example:

```
swy exec youtube.search_videos --body '{"query":"Elon Musk interview 2024"
```

6.5 Gmail — Answer Receipt Delivery

Purpose: Send Answer Receipts as emails, read email knowledge

Tool	Description	Policy
`gmail.send_email`	Send email	**Require Approval**
`gmail.list_messages`	List emails	Allow
`gmail.get_message`	Read email content	Allow

Used in: `src/lib/swytchcode/tools.ts` → Agent can send Answer Receipts

CLI example:

```
swy exec gmail.send_email --body '{"to":"user@example.com","subject":"Answ
# This will pause for approval per policies.json
```

6.6 Slack — Team Access

Purpose: Allow teams to query the twin via Slack channels

Tool	Description	Policy
`slack.post_message`	Post answer to channel	**Require Approval**
`slack.list_channels`	List available channels	Allow
`slack.get_channel_history`	Read channel messages	Allow

Used in: `src/lib/swytchcode/tools.ts` → Team interaction

CLI example:

```
swy exec slack.post_message --body '{"channel":"#mindcommit","text":"Accor
```

6.7 Telegram — Bot Interface

Purpose: Telegram bot for authorized users to query the twin

Tool	Description	Policy
`telegram.send_message`	Send bot message	**Require Approval**
`telegram.get_updates`	Get incoming messages	Allow

CLI example:

```
swy exec telegram.send_message --body '{"chat_id":"12345","text":"..."}'
```

6.8 Resend — Transactional Emails

Purpose: Send beautifully formatted Answer Receipt emails

Tool	Description	Policy
`resend.send_email`	Send transactional email	**Require Approval**

CLI example:

```
swy exec resend.send_email --body '{"to":"user@example.com","subject":"You
```

6.9 Firecrawl — Web Scraping

Purpose: Scrape public articles, research papers, blog posts

Tool	Description	Policy
`firecrawl.scrape_url`	Scrape a single URL	Allow (rate-limited)
`firecrawl.crawl_url`	Crawl a website	Allow (rate-limited)

Used in: `src/lib/swytchcode/sync-engine.ts` → Public content ingestion

CLI example:

```
swy exec firecrawl.scrape_url --body '{"url":"https://example.com/article"
```

6.10 Google Calendar — Timeline Context

Purpose: Add temporal context to knowledge commits

Tool	Description	Policy
`google_calendar.list_events`	List events	Allow
`google_calendar.create_event`	Create event	**Require Approval**

---

7. Configuration Files

`.swytchcode/tooling.json`

Location: `e:\hackthon\nsut hack\vibewrite\.swytchcode\tooling.json`

Defines which integrations and tools are available to MindCommit. This is the **trust boundary** — if a tool isn't listed here, the agent cannot use it.

`.swytchcode/policies.json`

Location: `e:\hackthon\nsut hack\vibewrite\.swytchcode\policies.json`

Defines **16 policy rules** that control agent behavior:

Rule Category	Count	Action
Allow reads	6 rules	`allow` — knowledge ingestion from Drive, Notion, GitHub, YouTube, Calendar, Firecrawl
Require approval	5 rules	`require_approval` — sending Gmail, Slack messages, Telegram, Resend, Calendar events
Deny destructive	1 rule	`deny` — ALL delete operations across ALL integrations
Rate limiting	2 rules	`rate_limit` — 10 sends/hour for communications, 50 scrapes/hour for Firecrawl

---

8. Backend Code — Where Swytchcode Is Used

File: `src/lib/swytchcode/tools.ts`

Purpose: Central registry of all Swytchcode tools and the execution function.

What it does:

- Defines all 10 integrations with their input schemas
- `executeSwytchcodeTool(toolName, input)` calls `swy exec <tool> --body '<json>'`
- Falls back to mock responses when CLI isn't available (demo mode)
- `getToolStatus()` reports which integrations are connected

Swytchcode features used: CLI execution, schema validation, mock fallback

---

File: `src/lib/swytchcode/sync-engine.ts`

Purpose: Polls external sources and creates Knowledge Commits from new/modified data.

What it does:

- `syncGoogleDrive()` → calls `googledrive.list_files` → downloads new files → creates commits
- `syncNotion()` → calls `notion.search_pages` → pulls page content → creates commits
- `syncGitHub()` → calls `github.search_repos` + `github.get_file_content` → creates commits
- `syncYouTube()` → calls `youtube.search_videos` + `youtube.get_captions` → creates commits
- Each sync function creates timestamped Knowledge Commits with source tracking

Swytchcode features used: Managed Auth (OAuth), retries, audit trail, consent ledger

---

File: `src/lib/swytchcode/config.ts`

Purpose: Exports Swytchcode configuration for the application.

What it does:

- Loads and exports `tooling.json` configuration
- Loads and exports `policies.json` configuration
- Provides helper functions for checking policy rules

File: `src/lib/agent/persona-agent.ts`

Purpose: Main agent that can invoke Swytchcode tools during chat.

Where Swytchcode is used:

- When the agent decides to send an Answer Receipt via email → `gmail.send_email`
- When the agent needs to look up additional context → `notion.search_pages`
- When the agent posts answers to team channels → `slack.post_message`

File: `src/app/api/ingest/route.ts`

Purpose: API route for ingesting knowledge from uploaded files.

Where Swytchcode is used:

- Future: trigger `syncGoogleDrive()` for Drive-based ingestion
- Future: trigger `syncNotion()` for Notion-based ingestion

---

9. Swytchcode Features Used

#	Feature	How MindCommit Uses It	Docs Reference
1	**CLI (`swy`)**	`swy init` , `swy get` , `swy add` , `swy exec` to set up and execute API tools	CLI Overview
2	**Runtime SDK (JS)**	Programmatic tool execution from Node.js backend — same execution engine as CLI	Runtime SDK
3	**Managed Auth**	OAuth for Google (Drive/Gmail/Calendar/YouTube), Notion, GitHub, Slack — zero hardcoded credentials	Managed Auth
4	**Schema Validation**	Every API call validated against the tool's JSON Schema before execution	Execution Pipeline
5	**Policy Engine**	`policies.json` controls allow/deny/require_approval/rate_limit for every tool	Policies
6	**Retries**	Automatic retry with exponential backoff on 429/500/503 errors	Retries
7	**Idempotency**	Prevents duplicate email sends, duplicate Slack posts, duplicate commit syncs	Idempotency
8	**Audit Trail**	Full log of every API call — viewable via `swy audit` — powers the Consent Ledger	Telemetry
9	**Dry-Run**	`swy exec --dry-run` tests tool calls without side effects	Execute Tools

#	Feature	How MindCommit Uses It	Docs Reference
10	`tooling.json`	Trust boundary — defines exactly which tools MindCommit can use	tooling.json
11	`policies.json`	Runtime guardrails — 16 rules controlling agent behavior	policies.json
12	**MCP Server**	Exposes tools to AI coding assistants (Cursor, Claude Code) via `swy mcp serve`	MCP Server
13	**Rate Limiting**	Policy-based rate limits (10 sends/hr, 50 scrapes/hr)	Production Guardrails
14	**Tool Discovery**	`swy search`, `swy discover` to find available integrations	Manage Integrations

---

10. MCP Server Integration

Swytchcode includes a built-in **Model Context Protocol (MCP) server** that exposes MindCommit's tools to AI coding assistants.

Start the MCP server

```
swy mcp serve
```

This exposes a 7-tool agent profile and 14-tool full profile, allowing tools like Cursor, Claude Code, Windsurf, and Copilot to interact with MindCommit's API tools.

Register with editors

```
swy init --editor=cursor    # For Cursor
swy init --editor=claude    # For Claude Code
swy init --editor=copilot   # For GitHub Copilot
```

---

11. Troubleshooting

Common issues

```
# Check Swytchcode health
swy doctor

# View audit trail
swy audit

# Check auth status
swy auth status

# Check project configuration
swy list      # List enabled tools
swy info <tool>  # Get tool details
```

Exit codes

Code	Meaning
0	Success
1	Auth failed
2	Tool not found
3	Blocked by policy
4	Invalid input
5	Provider error

Useful commands

```
# Test without side effects
swy exec gmail.send_email --dry-run --body '{"to":"test@example.com","subj

# Search for integrations
swy search webscraper
swy search email

# View tool schema
swy info gmail.send_email

# Plan execution (preview what would happen)
swy plan gmail.send_email
```